Encoder-Level Lipschitz Regularization in STEM-GNN: Preliminary Results

Internal Research Memo
Working draft for discussion

September 7, 2026

Abstract

STEM-GNN's finetune stage previously regularized only the linear classification head's Lipschitz constant (via a Frobenius-norm penalty on its weight matrix, `decoder_jac_coeff`), while the GNN encoder beneath it remains unfrozen and unconstrained during finetuning. This memo reports a preliminary comparison of that head-only penalty against a new matching penalty applied to the encoder backbone's own weight matrices (`encoder_lip_coeff`), added directly to `model/encoder.py`. Results are from a reduced-budget run on Cora and should be read as a first look, not a final claim.

1 Motivation

The finetune objective already included a Jacobian-style penalty on the decoder,

$$\mathcal{L}_{\text{head}} = \lambda_{\text{dec}} \|W_{\text{dec}}\|_F^2,$$

intended to bound the sensitivity of the final linear map to its input. However, during finetuning the encoder itself is not frozen (only the vector-quantizer is, by default) and carries no such constraint. An encoder that is free to amplify its input by an arbitrary factor can pass a perturbation through to the head no matter how tightly the head's own weight norm is controlled, so a head-only penalty bounds only the last link in the chain. We added an analogous penalty over the encoder's backbone weight matrices,

$$\mathcal{L}_{\text{enc}} = \lambda_{\text{enc}} \sum_{\ell} \|W_\ell\|_F^2,$$

summed over every weight matrix in the backbone's message-passing layers (SAGE/GAT/GCN/GIN, including per-expert weights when MoE layers are used), excluding biases and GAT's attention vectors. Both penalties are gated by independent coefficients and default to 0 (opt-in, no behavior change unless requested).

2 Experimental Setup

Honesty note on scope. This is a reduced-budget, single-dataset sanity check, not the project's standard evaluation protocol. It was run on a single CPU core (no GPU available in this environment), which made the project's default finetune budget (1000 epochs, early-stop patience 200, 10 splits per dataset, per config) impractical to run interactively. Numbers below should be treated as preliminary evidence about training dynamics, not final benchmark results.

- **Dataset:** Cora, node classification, all 10 splits shipped with the dataset (`finetune.py` always iterates over every provided split; there is no independent random seed here).
- **Backbone:** non-MoE GraphSAGE encoder (2 layers, hidden dim 768), loaded from the `default` pretrain checkpoint at epoch 50 (`ckpts/pretrain_model/default`).

- **Budget:** 60 finetune epochs, early-stop patience 30 – far short of the config-file default (1000 epochs / patience 200). Learning rate and all other hyperparameters use the `config/finetune.yaml` Cora defaults.
- **Coefficients:** a single untuned value, 10^{-3}, for `decoder_jac_coeff` and/or `encoder_lip_coeff` in each configuration below. No sweep over coefficient magnitude was performed.
- **Metric:** node-classification accuracy (%), mean ± standard deviation across the 10 splits, reported at each split's best validation epoch (the project's existing early-stopping selection rule).

3 Results

Configuration	λ_{dec}	λ_{enc}	Train	Val	Test
Baseline (no regularization)	0	0	98.43±4.25	76.28±3.34	75.87±2.10
Head-only (prior behavior)	10^{-3}	0	98.43±4.25	76.04±3.35	75.71±2.30
Encoder-only (new)	0	10^{-3}	99.64±0.86	75.98±2.82	75.78±1.94
Head + Encoder (new, combined)	10^{-3}	10^{-3}	99.93±0.21	75.96±2.84	76.03±0.88

4 Reading the Results

Clean-accuracy means are flat. All four configurations land within roughly one standard deviation of each other on test accuracy (75.7%–76.0%). At this coefficient and epoch budget, neither penalty moves mean clean accuracy on Cora in either direction. This is expected: Lipschitz regularization is a robustness/sensitivity control, not principally an accuracy lever, and clean i.i.d. accuracy is not the metric that should reveal its effect.

The head-only penalty changes nothing measurable. The head-only row is nearly identical to the baseline on every column, including the across-split standard deviation (train std 4.25 → 4.25, test std 2.10 → 2.30). This matches the original concern that motivated this change: constraining only the decoder's weight norm has little effect when the unconstrained encoder is free to reshape the input beforehand.

The encoder-side penalty visibly changes training dynamics. Both configurations with $\lambda_{\text{enc}} > 0$ show a sharp drop in across-split variance on train accuracy (std 4.25 → 0.86 → 0.21) and a smaller but consistent drop in test-accuracy variance (std 2.10–2.30 → 1.94 → 0.88) as the encoder penalty is added and then combined with the head penalty. In other words, encoder-level regularization made the 10 finetuning runs converge more consistently with each other, even though it did not change their average outcome at this coefficient. That is a real, measured difference in training behavior attributable specifically to constraining the encoder, since the head-only run does not reproduce it.

This does not yet test the actual robustness claim. The motivation for encoder-level Lipschitz control is bounded sensitivity to input perturbations (missing features, dropped or shifted edges, degree/homophily shift) – not clean-split variance. STEM-GNN already ships scripts for exactly this (`scripts/missing_feature.py`, `scripts/random_edge_drop.py`, `scripts/degree_shift_ood.py`, `scripts/homophily_shift_ood.py`). `degree_shift_ood.py` has since been run and is reported immediately below; the others were not run for this memo due to the single-CPU compute budget available. The variance reduction observed above is suggestive but is not, by itself, a substitute for that evaluation.

5 Testing the Propagation-Aware Hypothesis: Degree-Shift Robustness

Motivation. For `MySAGEConv`'s mean aggregation on a connected undirected graph, the aggregation operator is similar to a symmetric normalized adjacency matrix, so its spectral norm is ≤ 1: propagation never

amplifies a perturbation, at best it damps one. That damping shrinks toward 1 (vanishes) exactly under degree shift – low-degree and near-isolated nodes have little neighborhood to average over, so the aggregation step does little to attenuate whatever the encoder's weight matrices do to the input. This gives a concrete, structural reason to expect `encoder_lip_coeff` to matter specifically here, in a way it should not for clean i.i.d. accuracy (Section 3) or the feature-only shift tested in Section 7. This section tests that hypothesis directly rather than leaving it as a plausible-sounding argument.

Experimental setup. `scripts/degree_shift_ood.py` buckets Cora's nodes by degree into ID (15th–85th percentile), OOD-low, and OOD-high, trains only on ID nodes, and reports accuracy on all three buckets using the best-validation-epoch weights (this script snapshots and restores those weights explicitly, unlike `finetune.py`'s `EarlyStopping`, which only tracks the best metrics, not the weights themselves). Same reduced budget as the rest of this memo: the `default` non-MoE checkpoint at epoch 50, 60 finetune epochs, early-stop patience 30, and the same four $(\lambda_{\text{dec}}, \lambda_{\text{enc}})$ configurations as Section 3. `repeat=10` was set explicitly (the script's number of seeded runs). One practical note: this script imports `dataset/model/utils/task` as top-level packages but, unlike some sibling scripts, does not add the package root to `sys.path` itself, so it needs `PYTHONPATH=.../STEM-GNN/STEM-GNN` set manually to run at all.

Configuration	ID	OOD-low	OOD-high
Baseline (no regularization)	83.76±1.65	78.99±1.05	82.19±1.29
Head-only	83.63±1.94	78.77±0.93	82.00±1.27
Encoder-only	80.86±1.02	76.08±1.17	79.75±1.03
Head + Encoder (combined)	81.06±0.75	76.03±1.63	79.66±1.06

The hypothesis is not supported. Head-only regularization again changes nothing measurable, consistent with every earlier section. Encoder-only regularization at $\lambda_{\text{enc}} = 10^{-3}$, the same coefficient used everywhere else in this memo, drops *all three* buckets by roughly the same amount (ID −2.90, OOD-low −2.91, OOD-high −2.44 points) rather than selectively protecting the degree-shifted buckets. Checking the ID-minus-OOD gap directly, which is the quantity the propagation-damping argument actually predicts should shrink: the baseline's ID−OOD-low gap is 4.77 points; the encoder-only gap is 4.78 points – unchanged. The ID−OOD-high gap moves from 1.57 to 1.11 points, but that is well within the ∼1–1.3-point standard deviations on these numbers, not a signal. Combining with the head-only penalty does not rescue this: the combined row tracks the encoder-only row closely on every column. **This is a uniform capacity penalty, not an OOD-selective robustness benefit.**

A plausible reason this differs from the earlier sections' results: this script trains on only the ID bucket, roughly 946 nodes out of Cora's 2708 (versus the ∼1350-node train split used in Section 3's standard splits), so the same $\lambda_{\text{enc}} = 10^{-3}$ constrains a model fit to less data and may simply be too strong a coefficient for this specific training-set size, rather than the propagation-damping argument being wrong in principle. The natural next step is a coefficient sweep (e.g. 10^{-4}, 10^{-5}) on this exact degree-bucketed setup, to check whether a weaker constraint reveals the hypothesized OOD-selective effect once it is no longer swamped by generic under-fitting – that sweep has not been run yet.

6 Baseline Hyperparameter Optimization (Optuna and Hyperopt)

The comparison in Section 3 uses the un-tuned `config/finetune.yaml` defaults for Cora (learning rate 5×10^{-4}, dropout 0.15, batch normalization) for every configuration. Before trusting any accuracy difference between the regularization settings, it is worth first checking whether the *baseline* model (no head or encoder regularization, i.e. $\lambda_{\text{dec}} = \lambda_{\text{enc}} = 0$) is even reasonably tuned. We used Optuna to search the baseline's finetune-stage hyperparameters.

What was tuned. The pretrained encoder and vector-quantizer architectures are fixed by the loaded checkpoint (weight shapes must match the saved `state_dict`), so only finetune-stage hyperparameters that do not change layer shapes are searchable without re-pretraining:

- **Learning rate** (`finetune_lr`), log-uniform on $[10^{-4}, 10^{-2}]$;
- **Dropout** (`dropout`), uniform on $[0, 0.5]$;
- **Normalization mode** (`normalize`), categorical over {`none`, `batch`} (`layer` was excluded – it is accepted by the CLI but the encoder's forward pass applies `BatchNorm1d` regardless of which string is passed, so it is not actually a distinct option in the current code).

Both regularization coefficients were held fixed at $\lambda_{\text{dec}} = \lambda_{\text{enc}} = 0$ throughout this search: the goal here is a better-tuned *baseline*, not a joint search over baseline and regularization strength.

Implementation. The search was run twice, once with each library, over the identical search space and budget described below, so the two are directly comparable. `results/hpo/optuna_baseline.py` and `results/hpo/hyperopt_baseline.py` each build the same parameter dictionary that `finetune.py`'s command-line entry point would build (Cora defaults from `config/finetune.yaml`), then import and call `finetune.run(params)` directly (in-process, no subprocess spawn) so that each trial reuses the already-imported PyTorch/PyG modules. Optuna uses its default TPE sampler and persists every trial to an on-disk SQLite database (`results/hpo/optuna_baseline.db`); Hyperopt uses `tpe.suggest` and persists its `Trials` object to a pickle file (`results/hpo/hyperopt_baseline_trials.pkl`) after every trial. Both are resumable: if the process is interrupted, re-running the same script loads the existing state and only runs the remaining trials rather than starting over. *(This safeguard was added after two earlier long-running background experiments in this session were silently killed by the sandbox environment recycling mid-run, losing all unsaved progress.)*

A caught bug, noted for transparency. The first Hyperopt run re-created a fresh, identically-seeded random generator on every incremental `fmin` call instead of reusing one generator across calls. This made every "next" suggestion during the startup (random) phase identical to the first one, so all 12 trials silently evaluated the exact same hyperparameters instead of exploring the space – a real bug in this memo's driver script, not a property of Hyperopt. It was caught by inspecting the per-trial log before trusting any result (the repeated parameters and repeated accuracy were the tell), fixed by persisting the random generator's state alongside the trial history, and re-verified with a 4-trial smoke test showing genuine variation before the full run below was produced.

Search budget. Running the full 10-split, 1000-epoch protocol per trial is not tractable on the single CPU core available here, so the search phase uses a reduced budget: a single fixed Cora split (`finetune_seed=0`), 30 finetune epochs, early-stop patience 15, for 12 trials, optimizing validation accuracy on that split. The best-found hyperparameters from each library are then re-evaluated once under the same protocol as Section 3 (all 10 splits, 60 epochs, early-stop patience 30) to get a number directly comparable to the baseline row of that table.

Results. Optuna's 12-trial search found its best single-split validation accuracy (73.4%) at `finetune_lr` $\approx 3.47 \times 10^{-4}$, `dropout` ≈ 0.014, `normalize` = `none`. Hyperopt's 12-trial search (same space and budget, after the bug fix above) found its best single-split validation accuracy (75.0%) at `finetune_lr` $\approx 3.56 \times 10^{-4}$, `dropout` ≈ 0.094, `normalize` = `none`. The two libraries agree closely on learning rate (both land within 3% of 3.5×10^{-4}, versus the default 5×10^{-4}) and on turning batch normalization off, and disagree more on dropout (0.014 vs. 0.094, both well below the default 0.15) – consistent directional signal on two of three dimensions, with the third under-constrained at this trial budget. Re-evaluating each configuration under the full 10-split protocol (60 epochs, patience 30) gives:

Configuration	Train	Val	Test
Untuned defaults (Section 3 baseline)	98.43±4.25	76.28±3.34	75.87±2.10
Optuna-tuned baseline	98.50±4.27	76.36±2.85	75.91±1.78
Hyperopt-tuned baseline	98.50±4.27	76.18±3.10	75.94±2.03

Neither library meaningfully improves on the untuned baseline, and both land on essentially the same operating point. All three rows agree to within 0.2 points on every column mean – far inside one standard deviation – despite Optuna and Hyperopt proposing visibly different dropout values (0.014 vs. 0.094) to get there. Optuna's run shows the tightest val/test spread of the three (val std 2.85, test std 1.78); Hyperopt's sits between Optuna's and the untuned default (val std 3.10, test std 2.03). Practically, this says the `config/finetune.yaml` Cora defaults were already a reasonable operating point for this checkpoint and epoch budget – two independent search algorithms converged to the same conclusion – and the head-vs.-encoder regularization comparison in Section 3 is not an artifact of a poorly-tuned baseline.

6.1 Which hyperparameter actually matters? (48-trial follow-up)

The 12-trial searches above disagreed on *which* hyperparameter matters most: applying Optuna's importance evaluators to its own 12-trial study ranked `dropout` far ahead of the other two (fANOVA 0.703, MeanDecreaseImpurity 0.663), while the same random-forest-based analysis applied to Hyperopt's independent 12-trial study ranked `normalize` highest instead (0.646) with `dropout` a distant second (0.211). With only 12 points spread over 3 dimensions, `dropout` and `normalize` were not well-decorrelated in either sample, so a single search's importance ranking was not trustworthy.

To resolve this, the same Optuna study was extended from 12 to 48 trials (resuming the identical SQLite-backed study, same search space and per-trial budget as Section 6). At 48 trials, both importance evaluators now agree:

Parameter	fANOVA	MeanDecreaseImpurity
`dropout`	0.595	0.552
`normalize`	0.367	0.304
`finetune_lr`	0.038	0.144

Dropout is the most important of the three, with `normalize` a clear second and `finetune_lr` consistently least important – this resolves the 12-trial disagreement in Optuna's original favor, not Hyperopt's. The `normalize` effect is real and sizable on its own: across the 48 trials, `normalize=none` averages 73.4% single-split validation accuracy versus 66.8% for `normalize=batch` (37 vs. 11 trials), a 6.6-point gap. One caveat carries over even at n=48: `dropout` and `normalize` still show a moderate correlation (0.58) across the sampled trials, because Optuna's TPE sampler concentrates later trials near previously-promising regions rather than sampling independently – this is not a factorial design, just a much less noisy one than n=12.

But the extra tuning still doesn't beat the untuned baseline. Re-evaluating the new best configuration (`finetune_lr` $\approx 1.29 \times 10^{-4}$, `dropout` ≈ 0.088, `normalize` = `none`) under the full 10-split protocol gives train 98.50±4.27, val 76.02±3.23, test 75.87±2.04 – statistically indistinguishable from the untuned defaults (test 75.87±2.10) and no better than the earlier 12-trial-tuned result (test 75.91±1.78). This is the interesting part: `dropout` clearly dominates *variance in the single-split, 30-epoch search objective* used to rank configurations, but that dominance does not carry over to a measurable difference in final all-splits, 60-epoch test accuracy. Put plainly, correctly identifying the most important hyperparameter for the reduced-budget search protocol did not translate into a better model under the protocol this memo actually reports results on – a caution against reading too much into HPO importance analysis performed under a cheaper proxy budget than the final evaluation.

7 Test-Time Adaptation and Calibration Under Feature Shift

Motivation. A codebase exploration surfaced this as a natural complementary mechanism to the Lipschitz penalties above, not a replacement for them: the penalties in Section 3 act at *training* time, bounding how much the encoder and head are allowed to amplify their input; they say nothing about what happens at *inference* time on a graph that has already shifted. Two standard, well-established techniques address exactly that gap without touching training at all: adapting BatchNorm statistics to the test batch, and post-hoc temperature scaling.

Implementation. `Encoder.encode_with_bn_adaptation` (`model/encoder.py`) runs the forward pass with every `BatchNorm1d` layer normalizing against the current batch's own statistics rather than the stored running statistics from training ("prediction-time batch normalization," Nado et al. 2020; Schneider et al. 2020) – no weights change, and the running statistics are snapshotted and restored afterward, so this is a pure, reversible per-call adaptation. `utils/calibration.py` adds `fit_temperature` (post-hoc temperature scaling, Guo et al. 2017: fit a scalar T on held-out in-distribution logits, then divide test-time logits by T before softmax), `apply_temperature`, and `expected_calibration_error` (binned ECE). `task/node.py`'s `eval_node` gained `bn_adapt`, `temperature`, and `return_ece` keyword arguments (all default off, fully backward compatible), and `finetune.py`'s `run()` gained an `on_split_end` callback so an experiment can access the actually-trained model for extra post-hoc evaluation without duplicating the training loop.

Experimental setup. Same reduced budget as the rest of this memo: the `default` (non-MoE) checkpoint, 60 finetune epochs, early-stop patience 30, all 10 Cora splits, $\lambda_{\text{dec}} = \lambda_{\text{enc}} = 0$ (untuned-baseline architecture, since this section is about inference-time behavior, not the training-time penalties). **One deliberate deviation:** `normalize` is forced to `batch` for this experiment only, overriding Cora's `config/finetune.yaml` default of `none`. Under `normalize=none`, `Encoder.encode` never calls its `BatchNorm1d` layers at all, which would make BN adaptation a no-op by construction – turning batch normalization on is required to give it something to adapt. After training, each split's model is evaluated on: the clean graph; the same graph with a 50% missing-feature shift applied to validation+test nodes only (`scripts/ missing_feature.py`'s `_apply_missing_features`, `missing_prob=0.5`, `perturb="valtest"`); and that shifted graph again under BN adaptation, temperature scaling (fit once on the *clean* validation logits), and both combined.

Results. Mean ± std over the 10 splits, test split only:

Condition	Test Accuracy	Test ECE
Clean (no shift)	69.85±3.74	0.138±0.029
Shifted, no adaptation	62.40±4.75	0.125±0.031
Shifted, BN-adapted	66.48±3.89	0.132±0.031
Shifted, temperature-scaled	62.40±4.75	0.090±0.028
Shifted, BN-adapted + temperature	66.48±3.89	0.096±0.015

The fitted temperature averaged $T \approx 0.86 \pm 0.10$ across splits.

Reading the results. The 50% feature shift costs 7.5 points of test accuracy (69.85% → 62.40%). **BN adaptation recovers about 55% of that drop** (62.40% → 66.48%, +4.09 points) purely by renormalizing against the shifted batch's own statistics – no gradient steps, no labels, no change to any weight. **Temperature scaling recovers none of the accuracy** (62.40% → 62.40%, exactly unchanged, as expected: dividing logits by a scalar is a monotonic transform and cannot change which class has the highest score) but **meaningfully improves calibration** (0.125 → 0.090 ECE, a 28% relative reduction) even though T was fit only on clean, unshifted validation data – the miscalibration direction generalizes across this shift even though accuracy does not. Combining both gives BN's accuracy recovery alongside a comparable calibration improvement over the BN-only condition (0.132 → 0.096 ECE) and, notably, the tightest spread of any shifted condition (ECE std 0.015 vs. 0.028–0.031 elsewhere) – the two adaptations do not appear to interfere with each other. Two honest caveats: BN adaptation's own ECE is statistically flat relative to no adaptation (0.125 → 0.132, within noise) despite the large accuracy gain, so accuracy recovery and calibration are separate axes here, not one mechanism producing both; and all of this is demonstrated only under one shift type (feature dropout), one dataset, and the same reduced training budget used throughout this memo.

8 Next Steps

1. Run the existing OOD/perturbation scripts (missing-feature, random edge-drop, degree/homophily shift) comparing baseline vs. encoder-only vs. combined regularization, which is the setting that should

actually distinguish the two penalties.

2. Sweep `encoder_lip_coeff` (e.g. 10^{-4}–10^{-2}) instead of a single untuned value, since the variance-reduction effect observed here may strengthen or trade off against accuracy at a different magnitude. (A first attempt at this sweep was lost to the same sandbox interruption noted in Section 6 and has not yet been re-run.)

3. Once the Section 6 baseline is tuned, repeat the same Optuna search with λ_{enc} included as a search dimension, to compare a tuned regularized model against a tuned baseline rather than an untuned one.

4. Repeat on a second dataset (e.g. PubMed or WikiCS) and, compute permitting, restore the project's standard budget (1000 epochs, patience 200) to confirm the trend survives full convergence rather than being an artifact of early stopping at 60 epochs.